

Home Page

[<](#) [>](#) localhost:3000/

Discuss

Top Posts

Implementing Charts
By wpa 20 comments

Making an app
By Ramaz 35 comments

My project is done!
By mito 3 comments

Topics

- [javascript](#)
- [golang](#)
- [servers](#)
- [webdev](#)

Create a Topic

[<](#) [>](#) localhost:3000

Discuss

Top Posts

Implementing Charts
By wpa 20 comments

Making an app
By Ramaz 35 comments

My project is done!
By mito 3 comments

Create a Topic

Name

javascript

Description

laksjdlaksjdf

Topics

- [javascript](#)
- [golang](#)
- [servers](#)
- [webdev](#)

View a Topic

[<](#) [>](#) localhost:3000/topics/javascript

Discuss

javascript

Implementing Charts
By wpa 20 comments

Making an app
By Ramaz 35 comments

My project is done!
By mito 3 comments

JavaScript
Here you can discuss all things javascript. Share your projects, ask questions, and help others.

Create a Post

[<](#) [>](#) localhost:3000/topics/javascript/posts/new

Discuss

javascript

Implementing Charts
By wpa 20 comments

Making an app
By Ramaz 35 comments

My project is done!
By mito 3 comments

Create a Post

Title

Content

JavaScript
Here you can discuss all things javascript. Share your projects, ask questions, and help others.

View a Post

[<](#) [>](#) localhost:3000/topics/javascript/posts/123

Discuss

Implementing Charts
I'm trying to add a chart into my application, can anyone help me out?

Reply here

All 20 comments

Marcos

Have you tried using the Chart JS library?

Reply

mito

Yes, I tried that but I've been getting errors

Reply

Home Page

< > localhost:3000/

Discuss

Search

Sign Up

Top Posts

Implementing Charts

By wpa 20 comments

Making an app

By Ramaz 35 comments

My project is done!

By mito 3 comments

New Topic

Topics

- [javascript](#)
- [golang](#)
- [servers](#)
- [webdev](#)

PostList

View a Topic

< > localhost:3000/topics/javascript

Discuss

Search

Sign Up

javascript

Implementing Charts

By wpa 20 comments

Making an app

By Ramaz 35 comments

My project is done!

By mito 3 comments

Create Post

Javascript

Here you can discuss all things javascript. Share your projects, ask questions, and help others.

View a Post

< >

Discuss

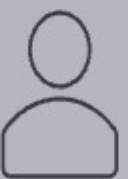
Implementing Charts

I'm trying to add a chart into my application, can anyone help me out?

All 20 comments



Marcos
Have you tried using the Chart JS library?
Reply



mito
Yes, I tried that but I've been getting errors
Reply

PostShow

CommentCreateForm

CommentList

CommentShow

View a Topic



localhost:3000/topics/javascript

Discuss

Search

Sign Up

javascript

Create Post

Implementing Charts

By wpa 20 comments

Making an app

By Ramaz 35 comments

My project is done!

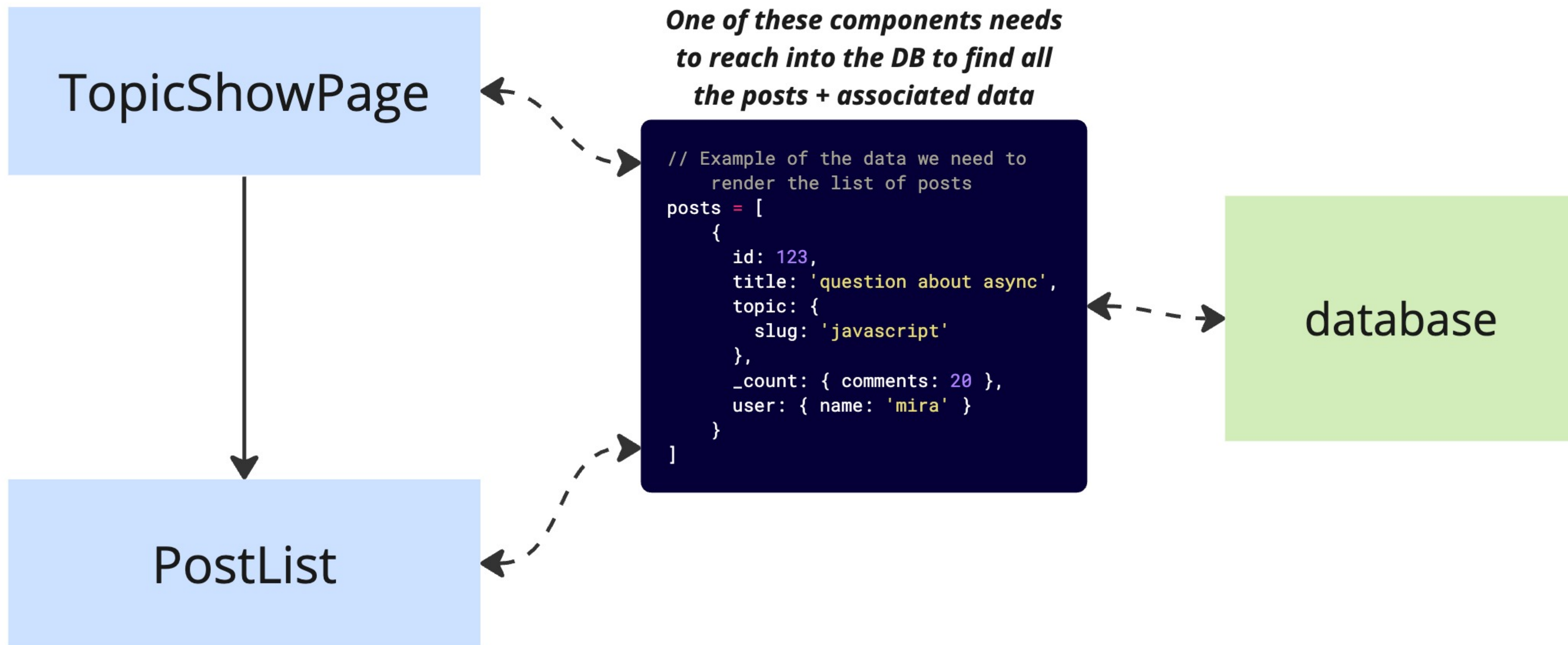
By mito 3 comments

JavaScript

Here you can discuss all things javascript. Share your projects, ask questions, and help others.

PostList





Which component should make the query?

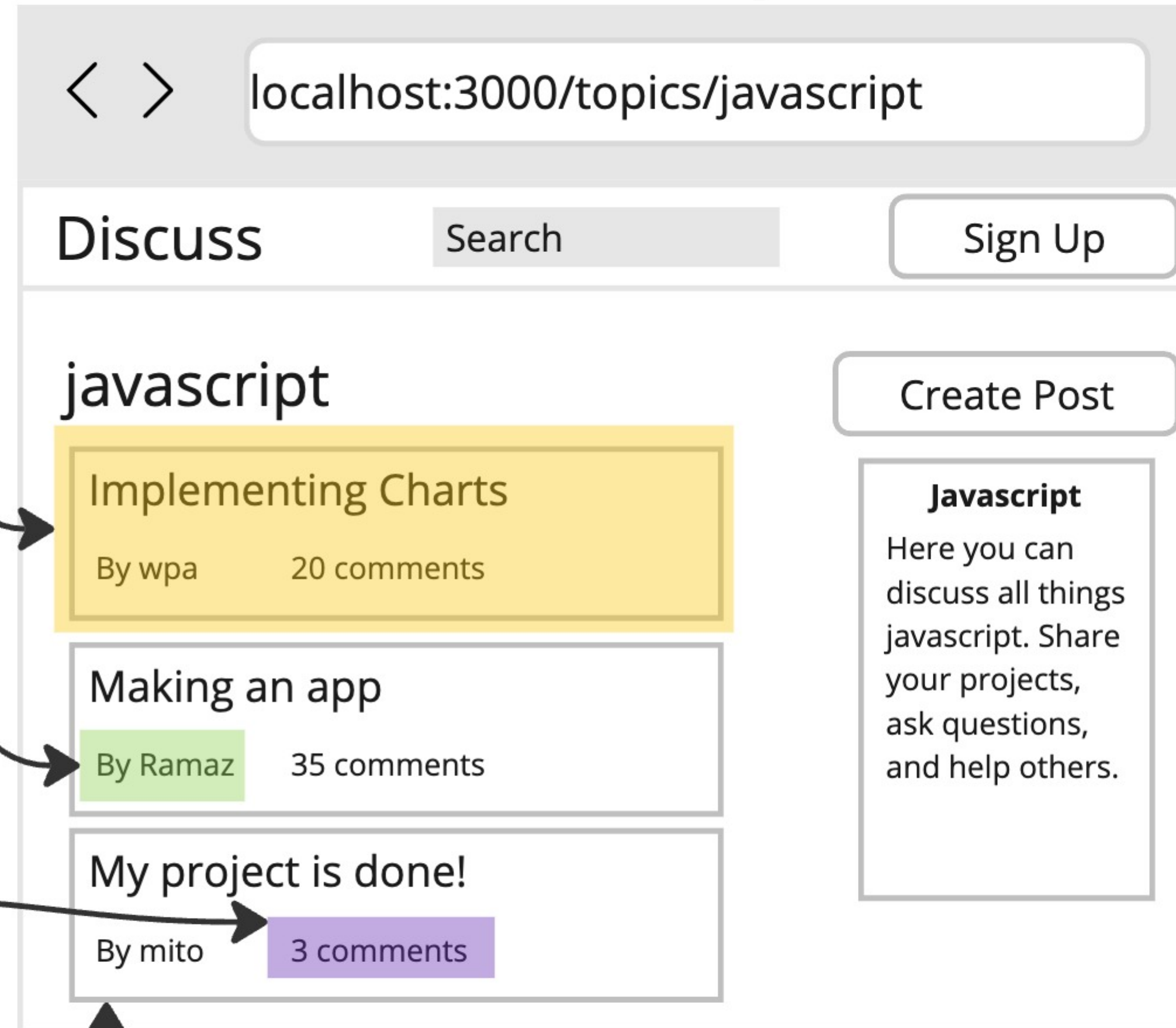
View a Topic

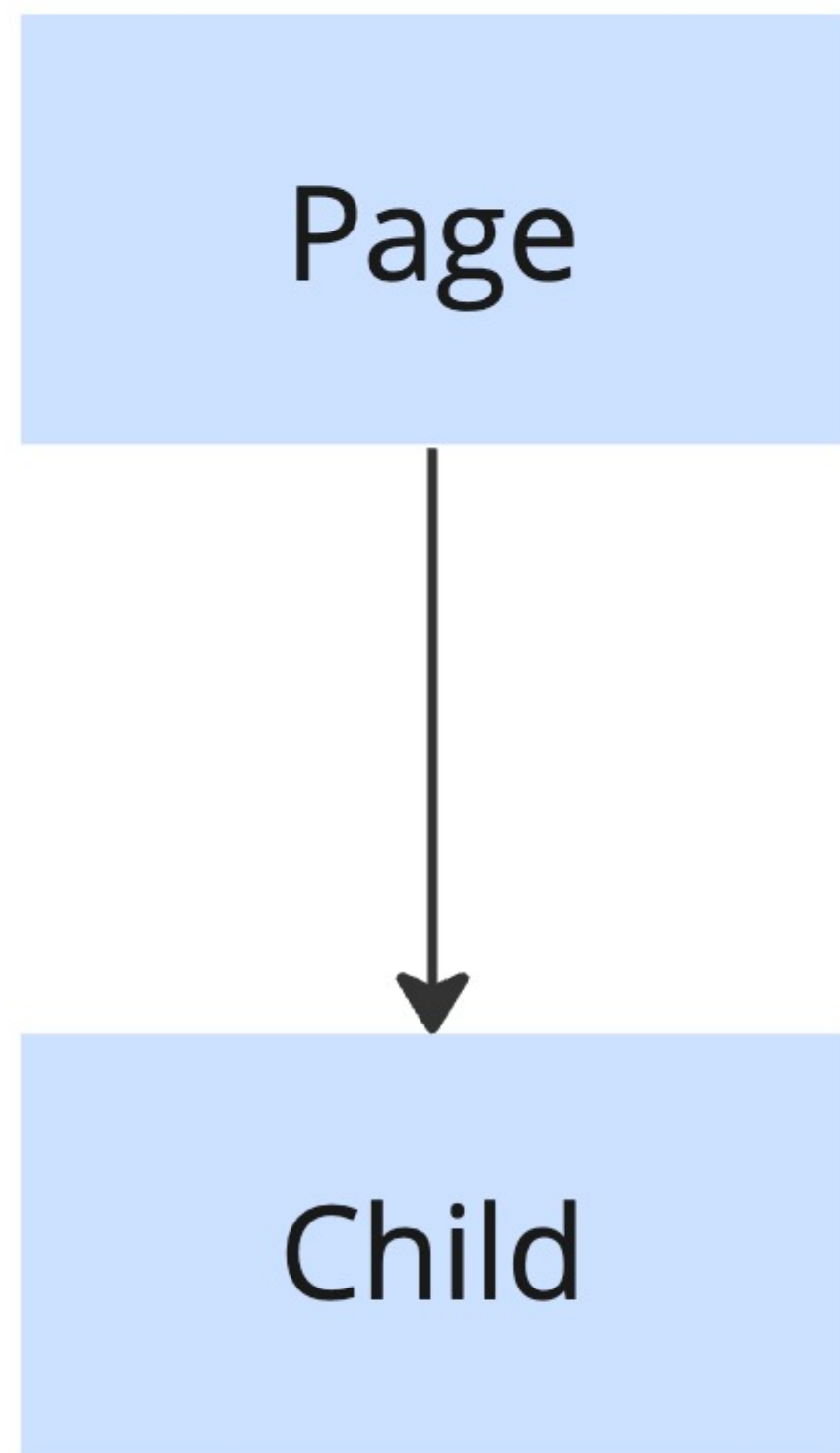
Need all the posts that belong to this topic

Need to get the **'name'** of the user who created each post

Need to get the **number of comments** for each post

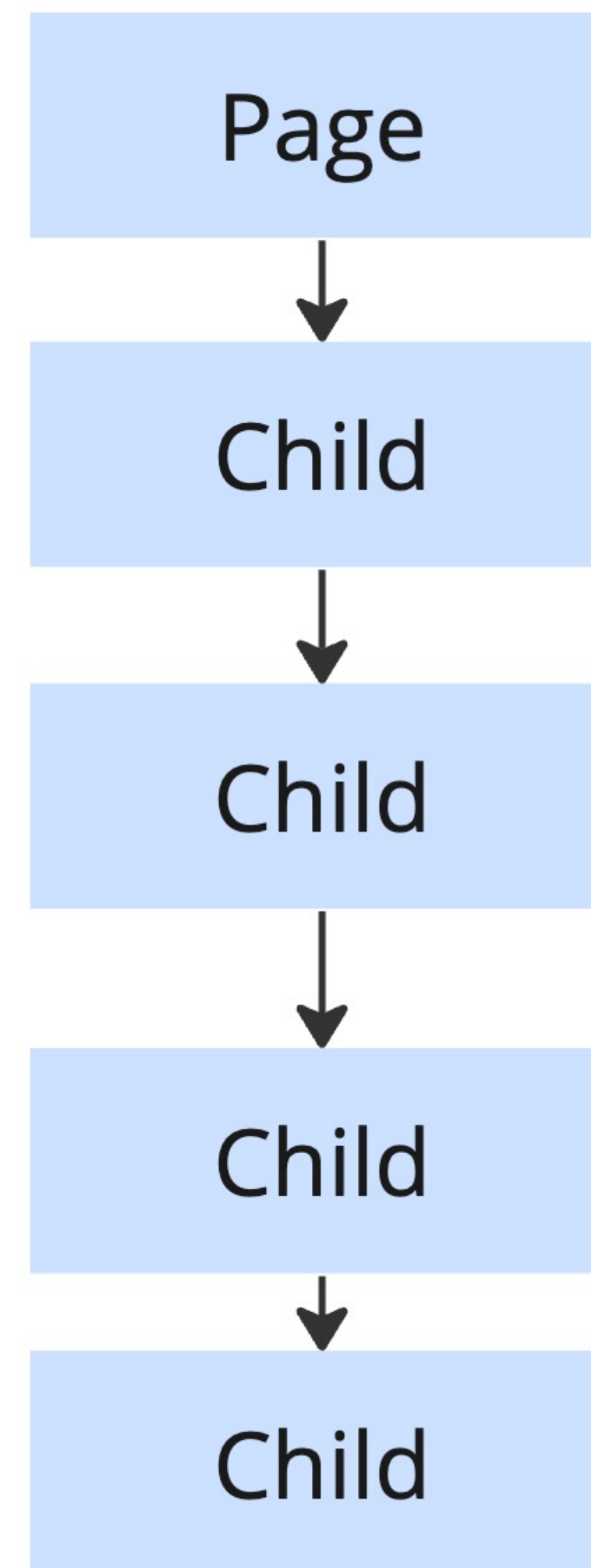
When a user clicks on a post, they need to navigate to the PostShowPage, at a route of **/topics/[slug]/posts/[post.id]**.
That means we also need to know the **slug** of the topic that this post is connected to





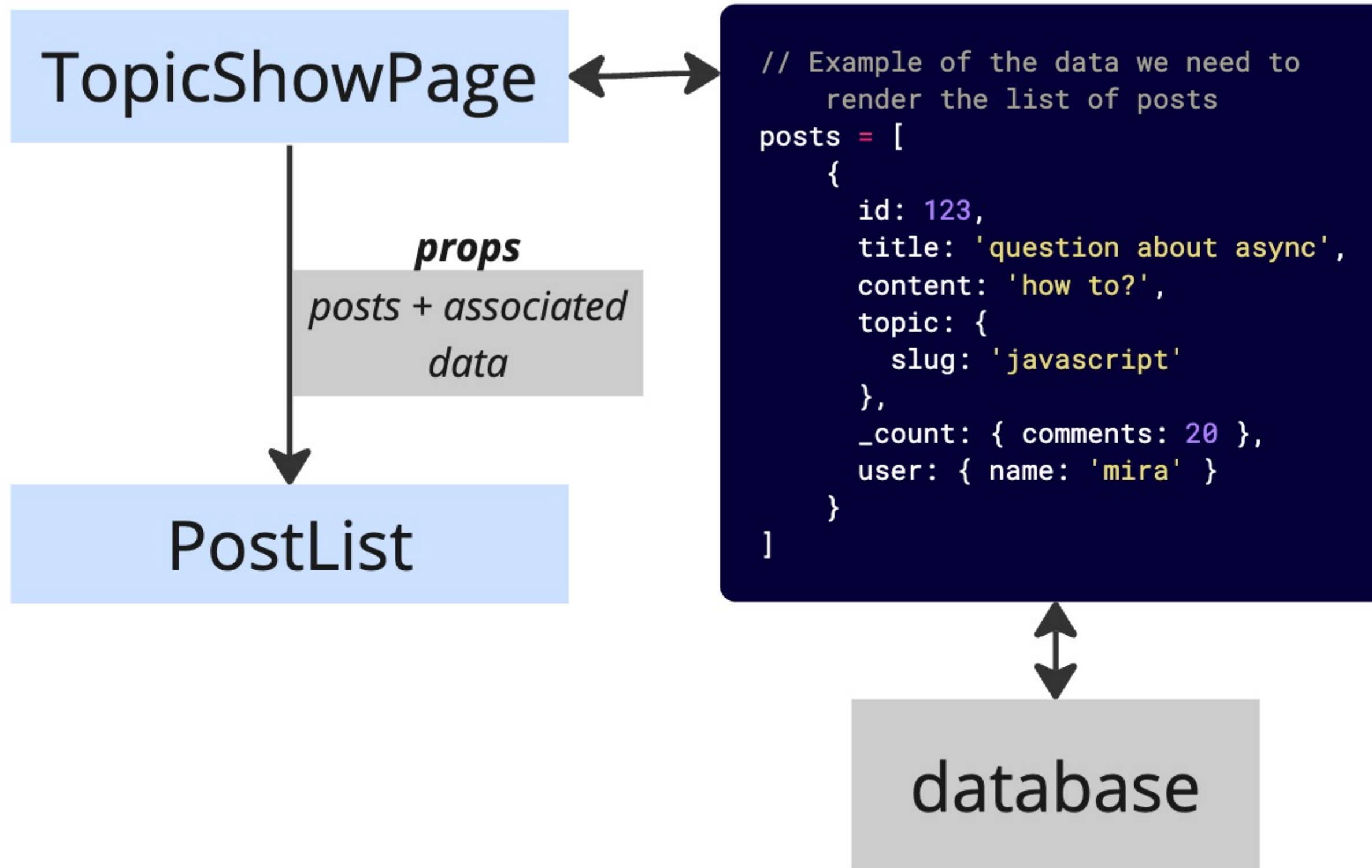
I'm going to refer to data fetching in the 'Page' vs a 'Child'

This discussion is really about prioritizing data fetching ***higher up*** your component hierarchy ***vs lower down***



Option #1

Page component gets data, passes it to child



Option #2

Child component fetches its own data

TopicShowPage



PostList



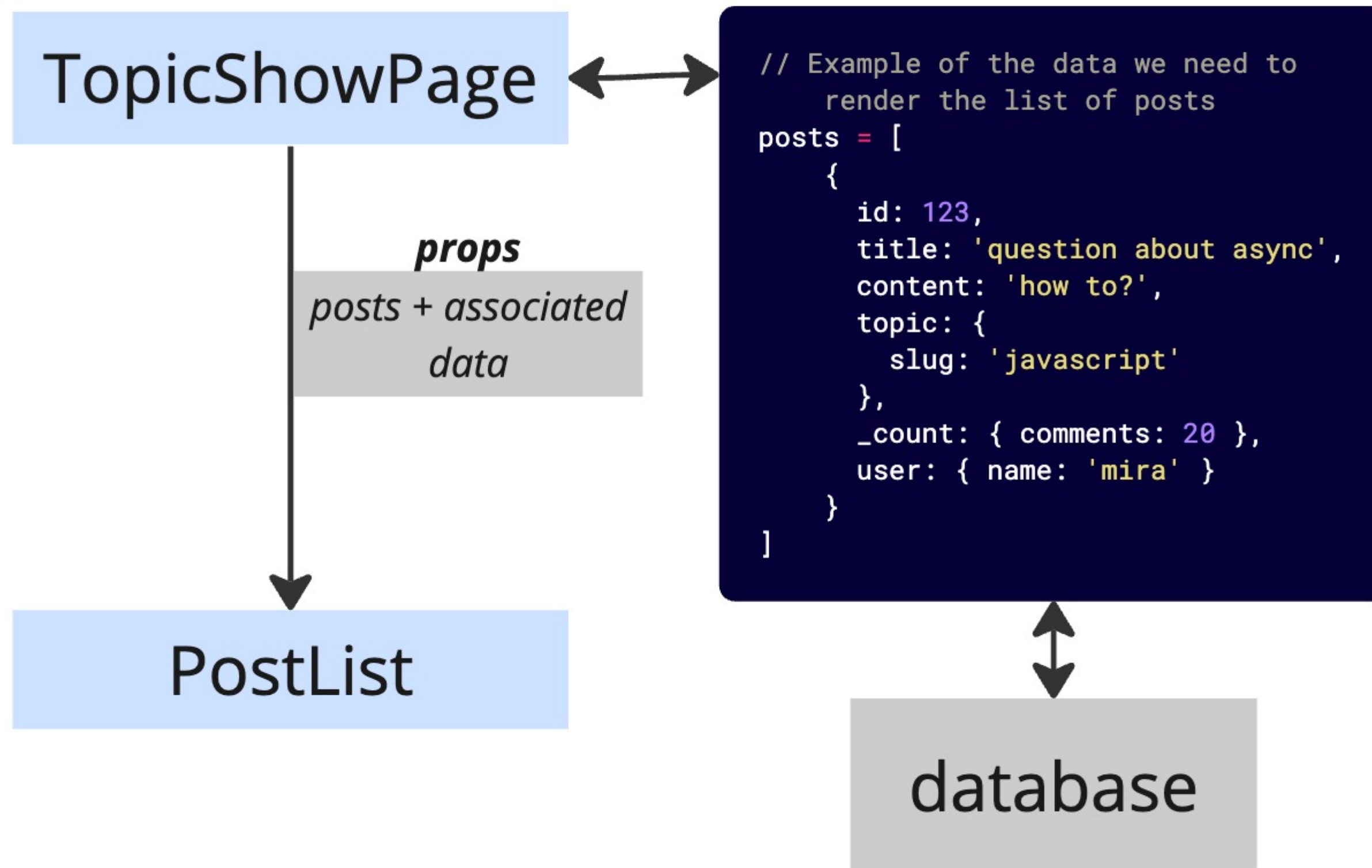
```
// Example of the data we need to
// render the list of posts
posts = [
  {
    id: 123,
    title: 'question about async',
    content: 'how to?',
    topic: {
      slug: 'javascript'
    },
    _count: { comments: 20 },
    user: { name: 'mira' }
  }
]
```



database

Option #1

Page component gets data, passes it to child



Easy to see what data a route needs to be displayed

Easier to make the child component reusable

Easier to avoid 'n+1' query issues

Can lead to overfetching of data

Can lead to duplicate code in other Pages using the child component

Sometimes annoying to write the interface for complex query data

Option #2

Child component fetches its own data

TopicShowPage



PostList



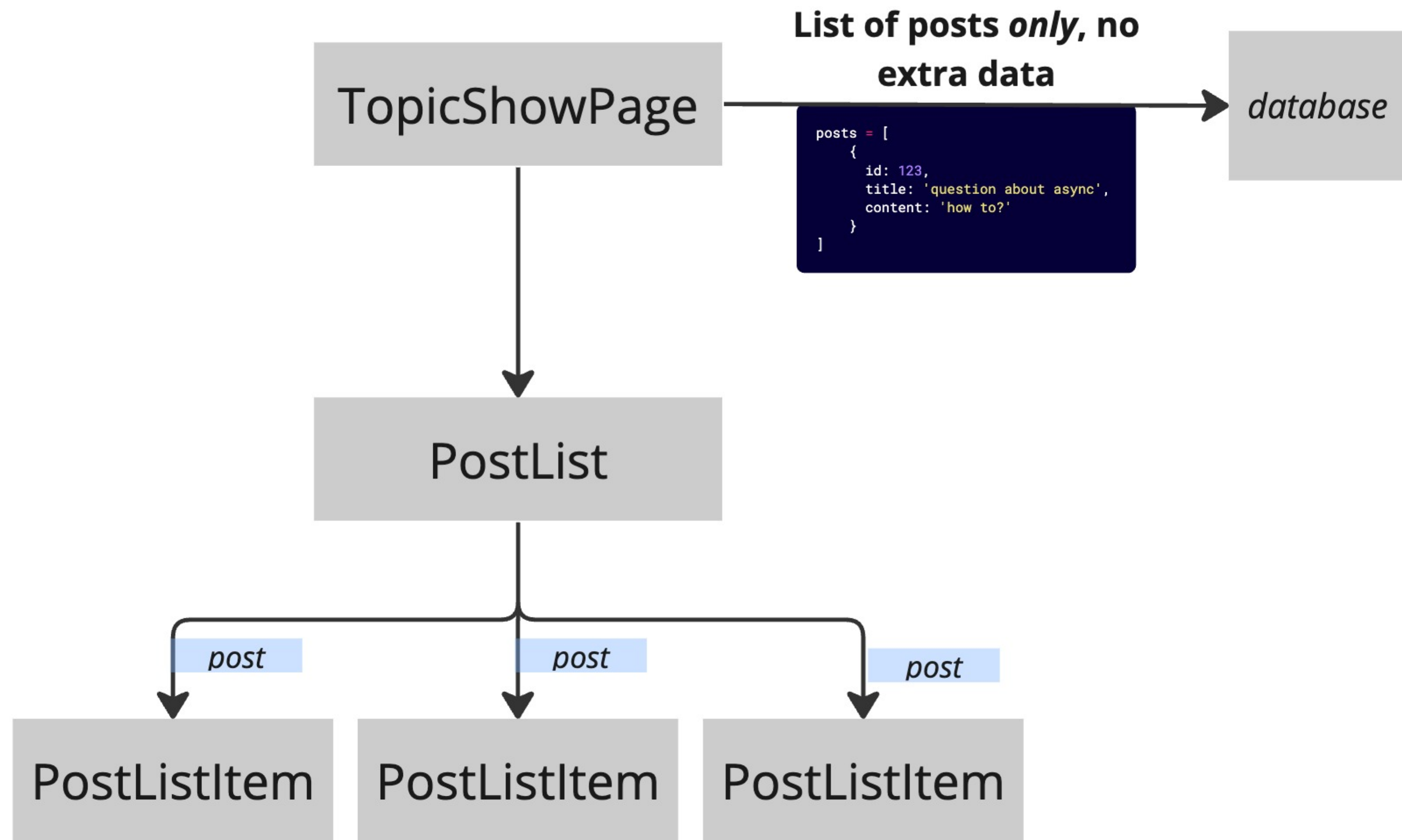
```
// Example of the data we need to
render the list of posts
posts = [
  {
    id: 123,
    title: 'question about async',
    content: 'how to?',
    topic: {
      slug: 'javascript'
    },
    _count: { comments: 20 },
    user: { name: 'mira' }
  }
]
```

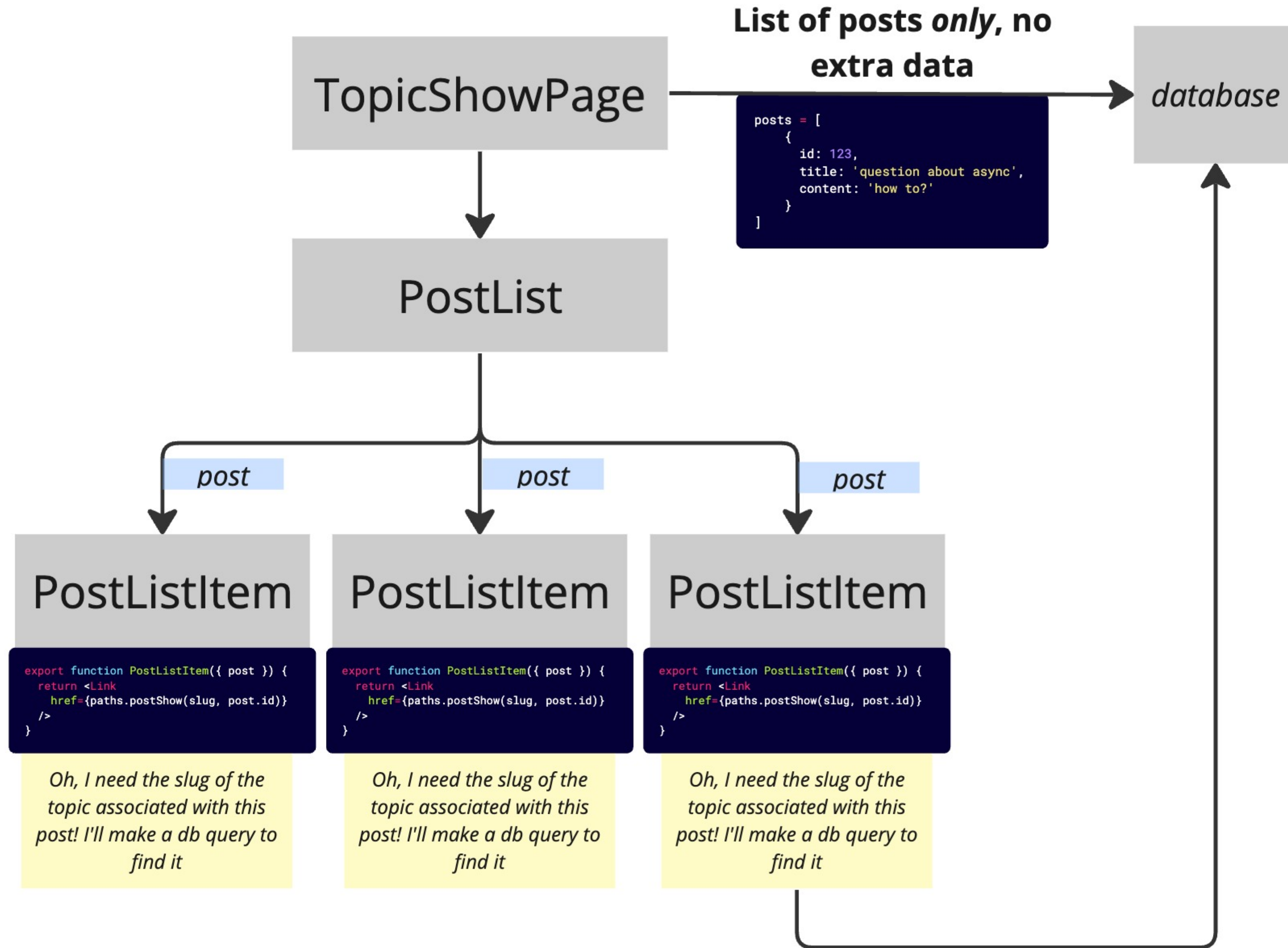


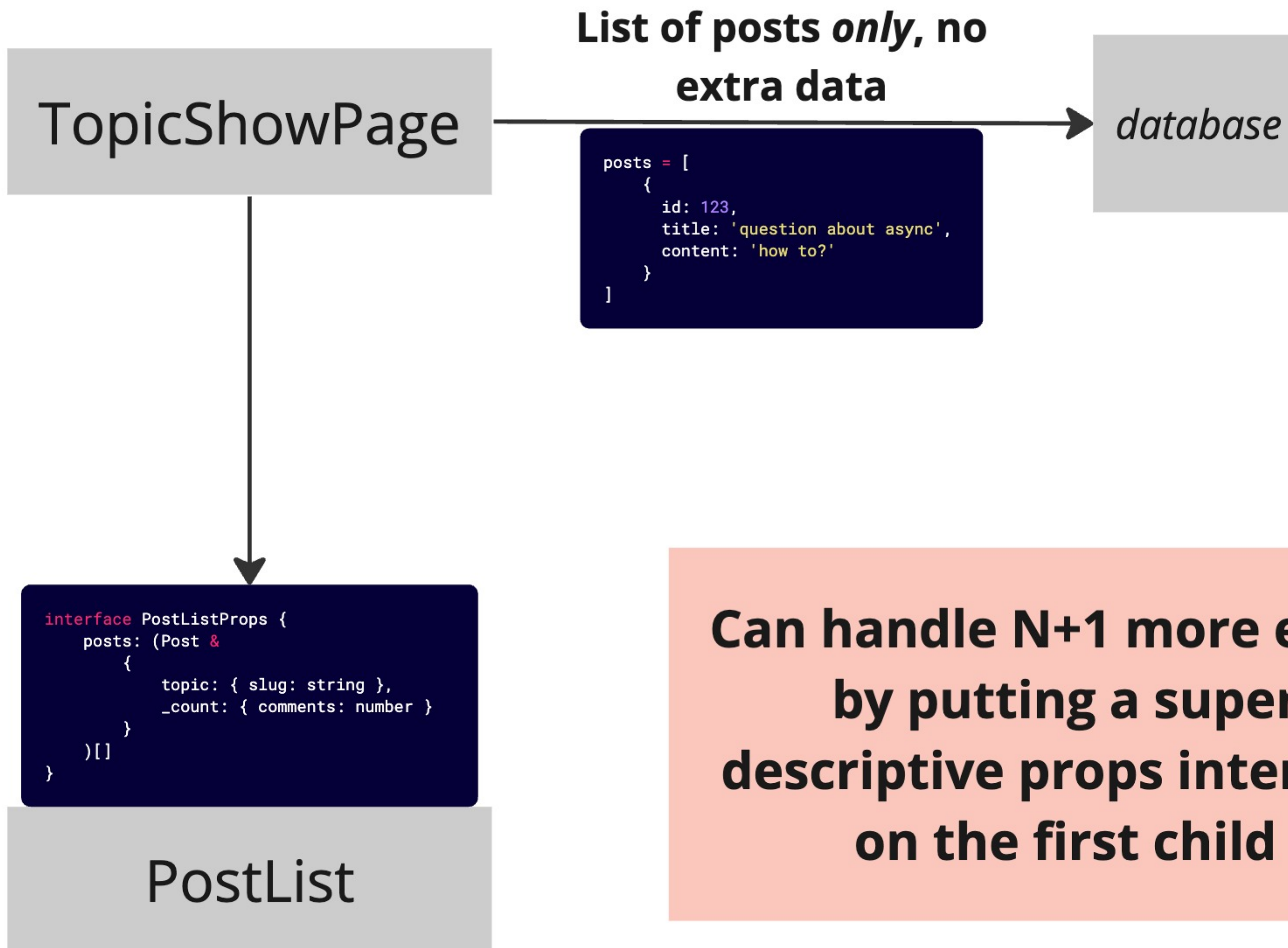
database

Easier to build 'skeleton' loading pages

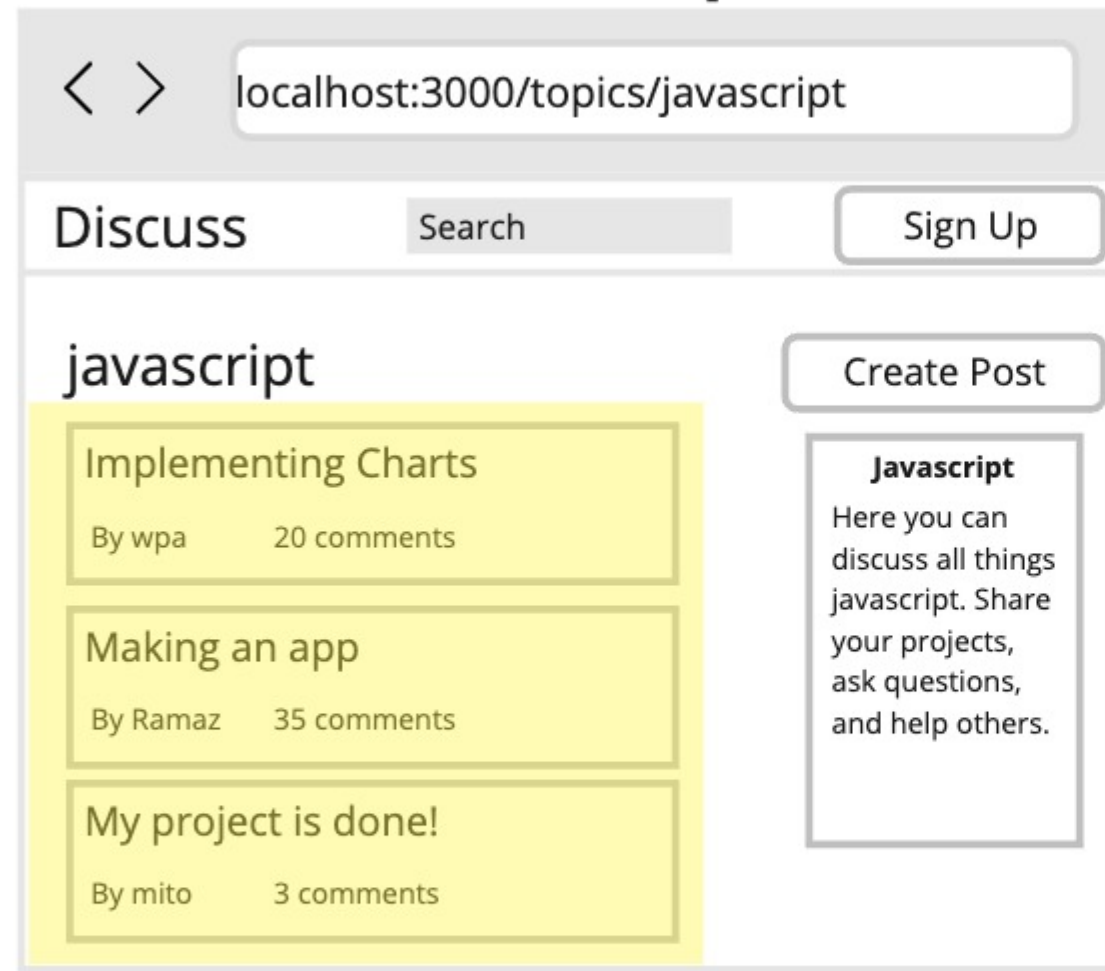
Child components implementation is *locked in*



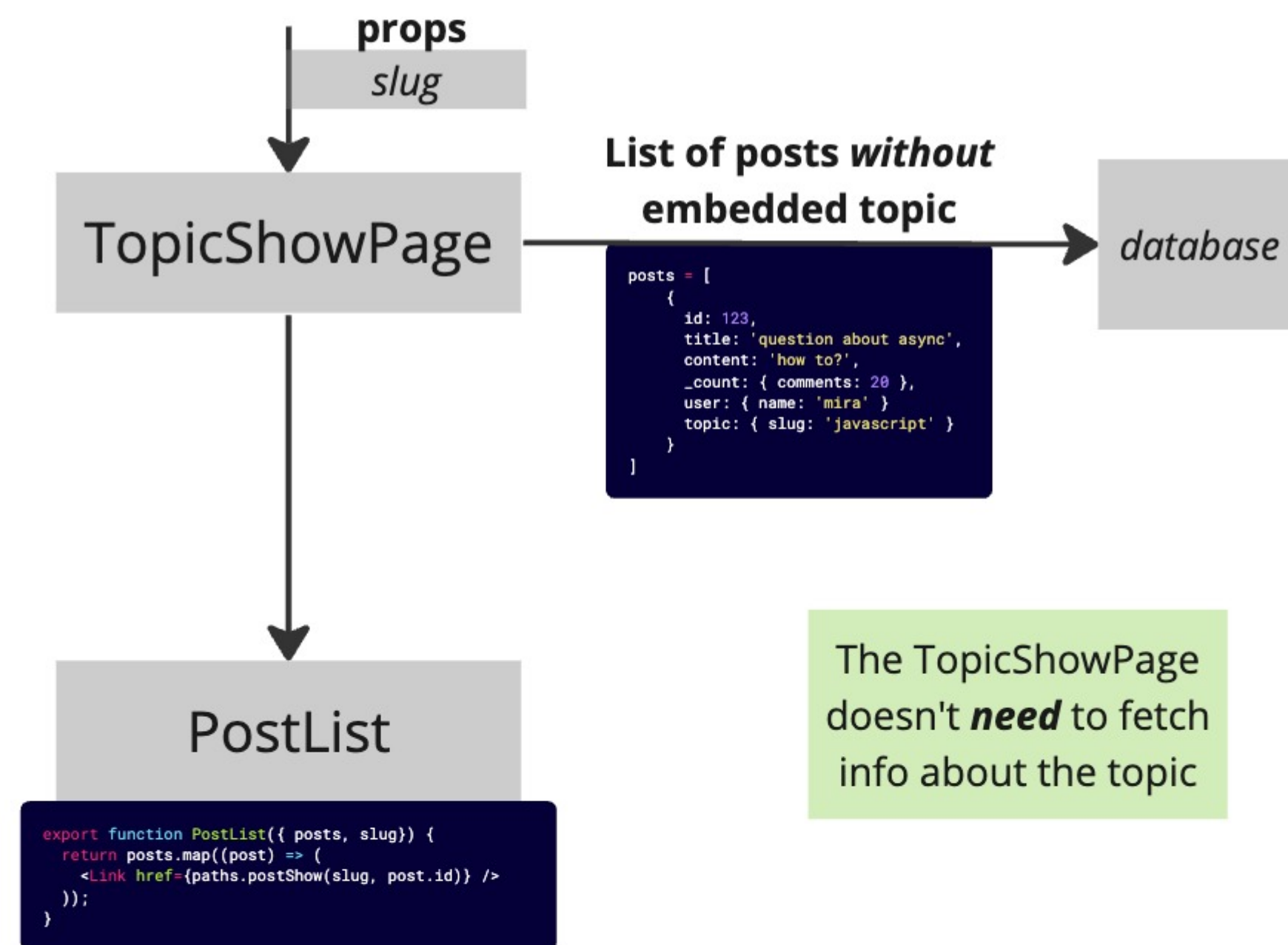




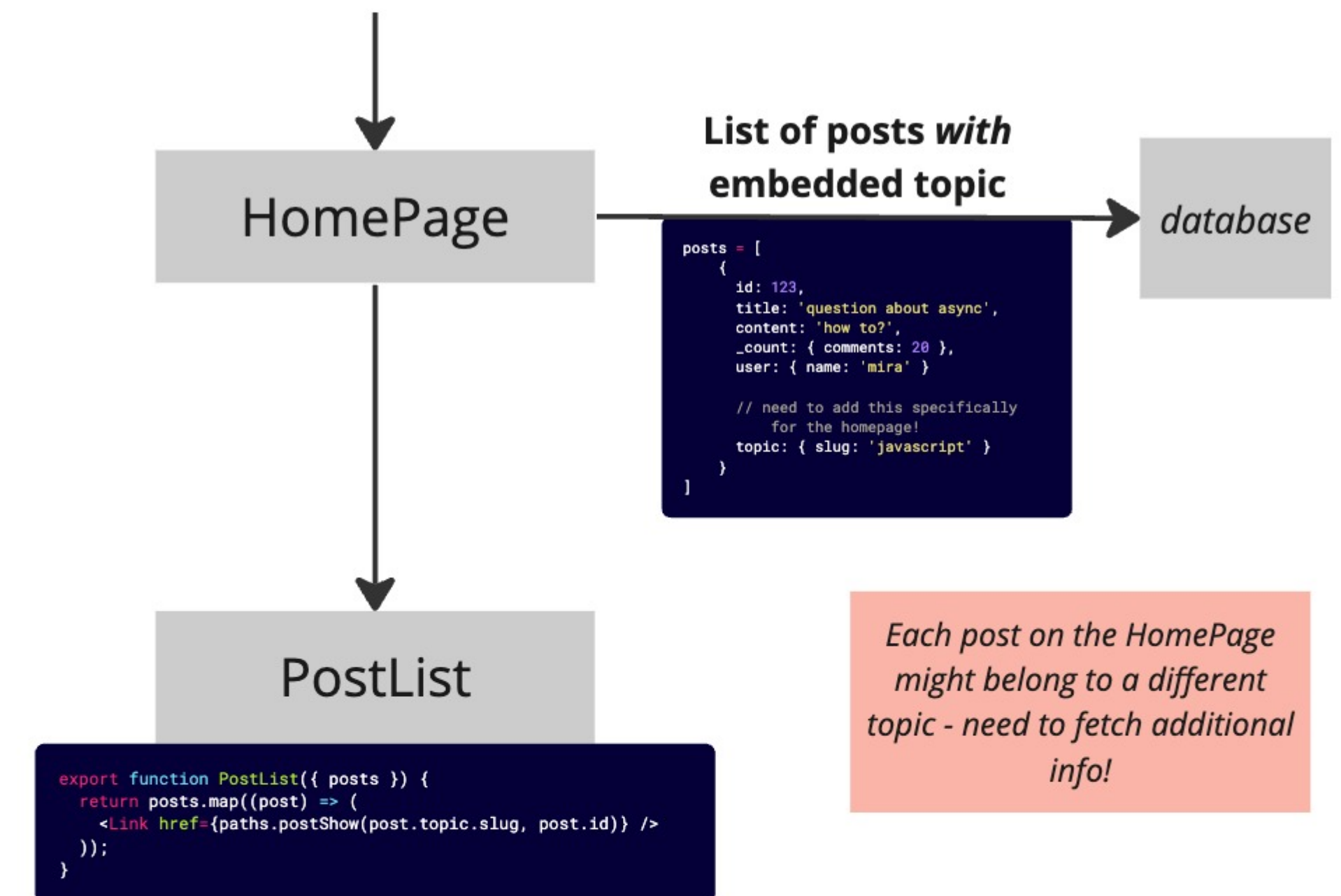
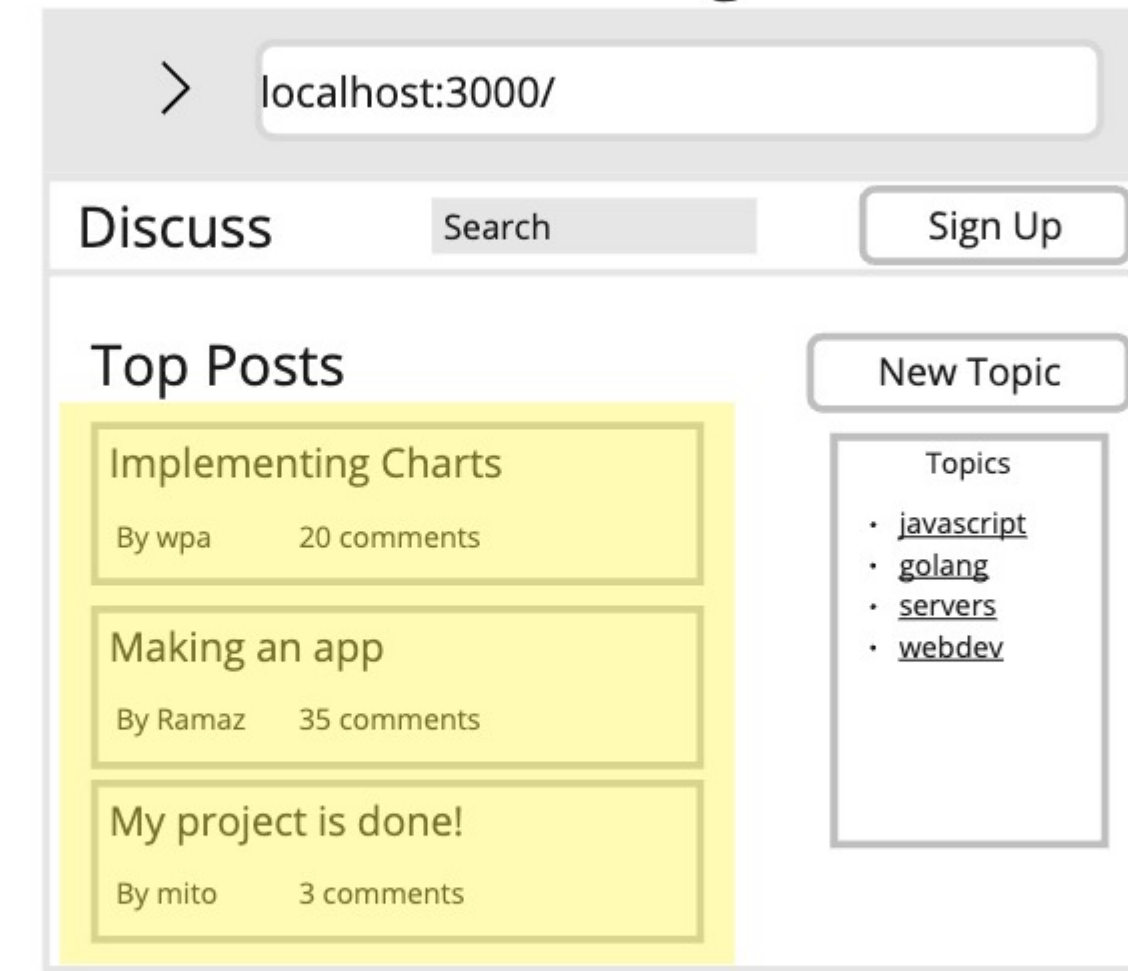
View a Topic



To link to a particular post, we need a **postId** *and* the slug of the topic the post belongs to



Home Page



Prisma automatically creates interfaces for us to describe the structure of data we pull out of the database

Definition of what a '**Topic**' is

```
type Topic = {  
  id: string;  
  slug: string;  
  description: string;  
  createdAt: Date;  
  updatedAt: Date;  
}
```

Definition of what a '**Post**' is

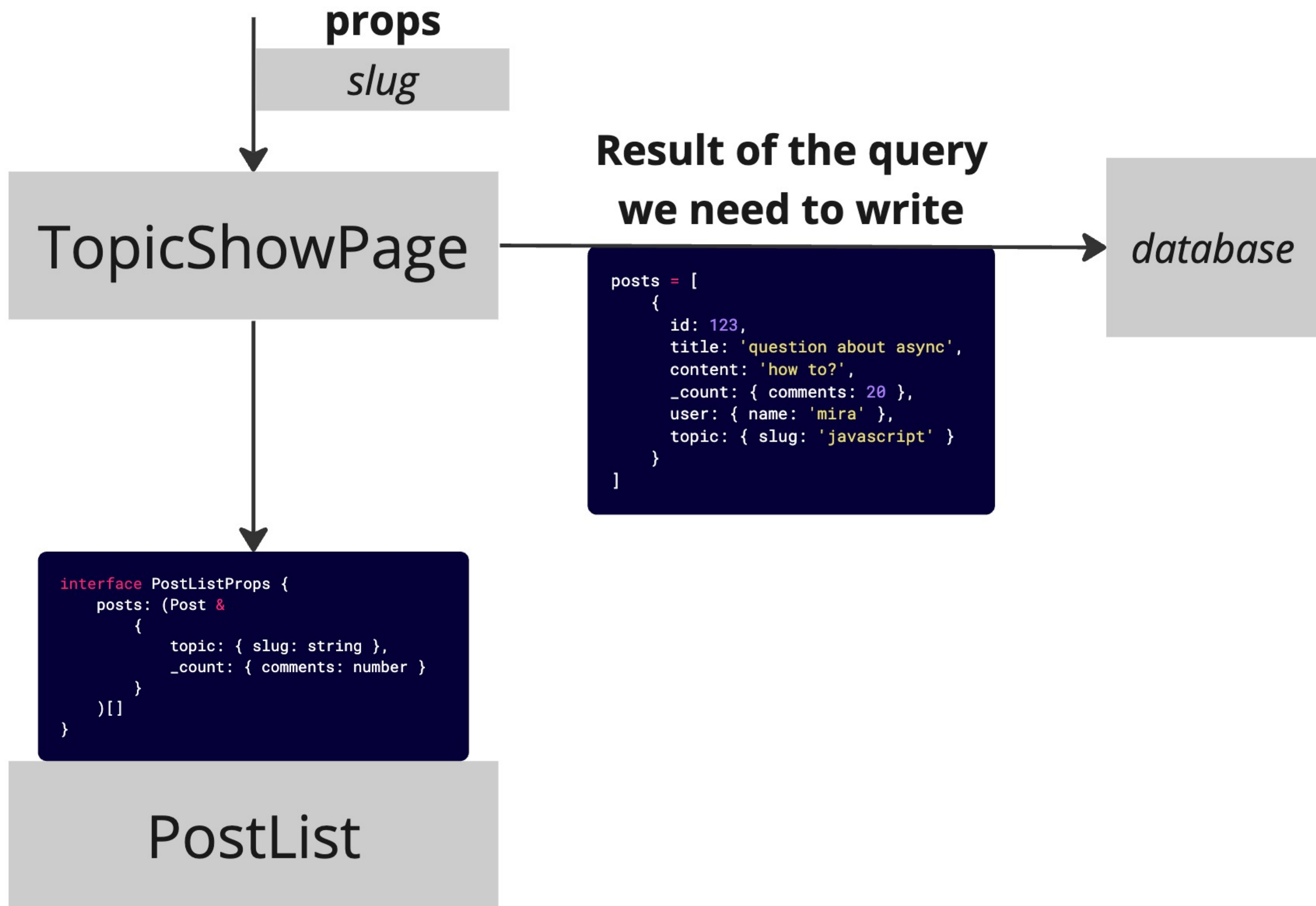
```
type Post = {  
  id: string;  
  title: string;  
  content: string;  
  userId: string;  
  topicId: string;  
  createdAt: Date;  
  updatedAt: Date;  
}
```

Definition of what a '**User**' is

```
type User = {  
  id: string;  
  name: string | null;  
  email: string | null;  
  emailVerified: Date | null;  
  image: string | null;  
}
```

Definition of what a '**Comment**' is

```
type Comment = {  
  id: string;  
  content: string;  
  postId: string;  
  userId: string;  
  parentId: string | null;  
  createdAt: Date;  
  updatedAt: Date;  
}
```



View a Topic During Load

[<](#) [>](#) localhost:3000/topics/javascript

Discuss [Sign Up](#)

javascript

[Create Post](#)

Javascript

Here you can discuss all things javascript. Share your projects, ask questions, and help others.



View a Topic Load Complete

[<](#) [>](#) localhost:3000/topics/javascript

Discuss [Sign Up](#)

javascript

Implementing Charts
By wpa 20 comments

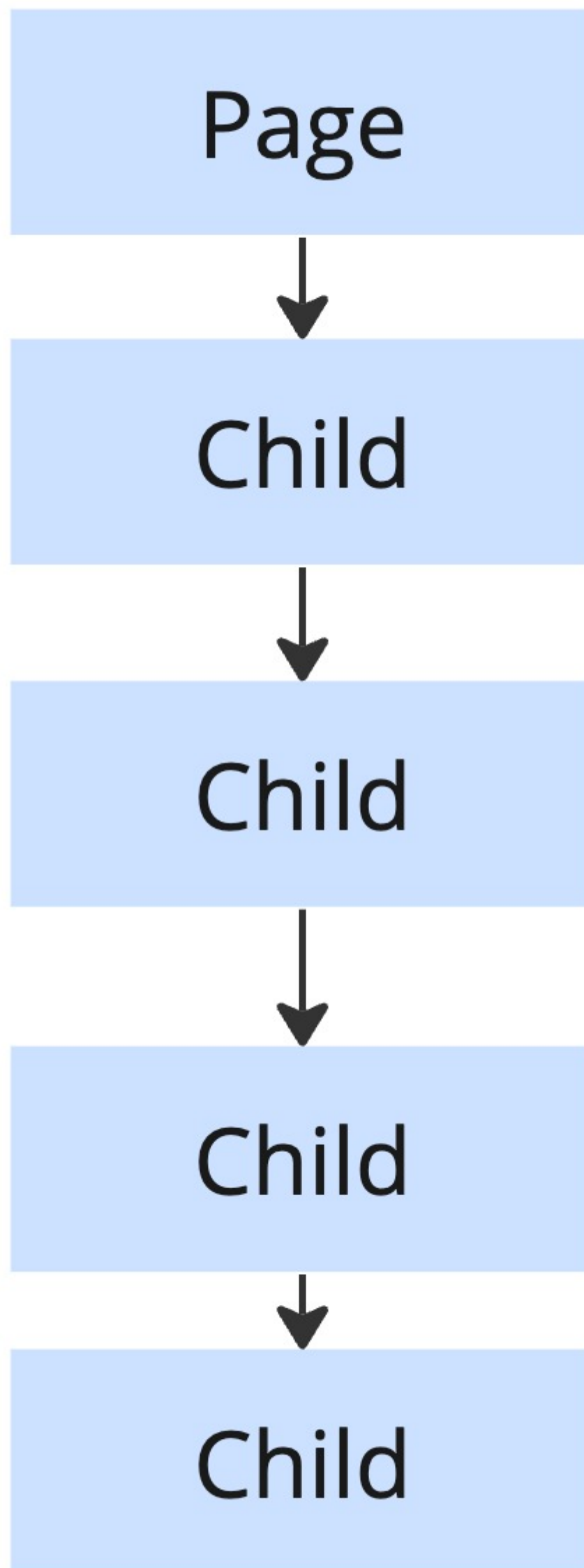
Making an app
By Ramaz 35 comments

My project is done!
By mito 3 comments

[Create Post](#)

Javascript

Here you can discuss all things javascript. Share your projects, ask questions, and help others.



Option #1

Data fetching towards top of hierarchy

Easier to make child components reusable, but slower page load speed

Option #2

Data fetching towards bottom of hierarchy

Faster page load, but components less reusable

Post Query File

```
type PostWithData = (Post & {  
  topic: { slug: string },  
  _count: { comments: number },  
  user: { name: string }  
})  
  
export function fetchPostsBySlug(slug: string): Promise<PostWithData[]> {  
  
}  
  
export function fetchTopPosts(): Promise<PostWithData[]> {  
  
}
```

**Separate file that lists all the queries
that can provide data to 'PostList'**

**PostList expects to receive a function
that will return 'PostWithData[]'**

Post Query File

```
interface PostListProps {  
  fetchPosts: () => Promise<PostWithData[]>  
}
```

PostList

```
type PostWithData = (Post & {  
  topic: { slug: string },  
  _count: { comments: number },  
  user: { name: string }  
})  
  
export function fetchPostsBySlug(slug): Promise<PostWithData[]> {  
    
}  
  
export function fetchTopPosts(): Promise<PostWithData[]> {  
    
}
```

Parent components can decide *what* data to fetch, children fetch it

TopicShowPage

```
import { fetchPostsBySlug } from 'queries/posts';
import PostList from './post-list';

export default function TopicShowPage({ params: { slug } }) {
  return <PostList
    fetchData={() => fetchPostsBySlug(slug)}
  />
}
```

HomePage

```
import { fetchTopPosts } from 'queries/posts';
import PostList from './post-list';

export default function HomePagae() {
  return <PostList
    fetchData={fetchTopPosts}
  />
}
```

PostList

```
export default function async PostList({ fetchData }) {
  const posts = await fetchData();

  return posts.map(() => ...)
}
```

Option #1.5

Define 'query' functions, pass them from the parent to child, child calls to fetch its own data without knowing the query's implementation

queries/posts.ts

```
import type { PostWithData } from 'post-list';

type PostWithData = (Post & {
  topic: { slug: string },
  _count: { comments: number },
  user: { name: string }
})

export function fetchPostsByTopic(slug: string):
  Promise<PostWithData[]> {
  return /*
    query to get posts related to the given topic
  */
}

export function fetchTopPosts(): Promise<PostWithData>
{
  return /*
    query to get top posts
  */
}
```

TopicShowPage

fetchPosts

```
interface PostListProps {
  fetchPosts: () => Promise<PostWithData[]>;
}
```

PostList

HomePage

fetchPosts

```
interface PostListProps {
  fetchPosts: () => Promise<PostWithData[]>;
}
```

PostList

Option #1.5

Define 'query' functions,
pass them from the parent
to child, child calls to fetch its
own data without knowing
the query's implementation

Helps avoid n+1 query issues

Easy skeleton loading pages

Easier to make the child component reusable

Less chance of duplicating queries across the project

